

LAPORAN PRATIKUM STRUKTUR DATA

Queue



Disusun Oleh:

Shofwatul Hasanah Sarma

NIM 2511533015

Dosen Pengampu: Wahyudi. DR, S.T.M.T

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG

2026

KATA PENGANTAR

Puji dan syukur kita panjatkan kehadirat Allah yang Maha Esa karena telah memberikan nikmat sehat jasmani dan rohani sehingga saya bisa menyelesaikan laporan ini. Di dalam laporan ini, terdapat hasil praktikum, pembahasan serta kesimpulan yang saya susun secara sistematis. Saya sebagai penulis sekaligus penyusun laporan ini sangat berterimakasih kepada Bapak Wahyudi selaku dosen dalam mata kuliah praktikum Struktur Data Departemen Informatika, Universitas Andalas, yang telah menuntun dan mengajarkan saya mata kuliah ini.

Dalam penulisan laporan praktikum ini, saya mendapatkan suatu pelajaran baik serta mendapatkan pengetahuan tentang praktikum tersebut. Saya sebagai penulis dalam laporan ini masih dalam proses belajar, baik dalam tata penulisan laporan yang benar maupun dalam memahami penyajian laporan yang sesuai kaidah. Untuk itu saya berharap mendapatkan saran-saran yang membangun sehingga dapat meningkatkan pengetahuan saya dan dapat membuat isi laporan yang lebih baik, mendidik dan dapat dipertanggungjawabkan.

Padang, 2026

Shofwatul Hasanah Sarma

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI.....	ii
BAB I.....	3
PENDAHULUAN	3
1.1 Latar Belakang.....	3
1.2 Tujuan Praktikum	3
1.3 Manfaat Praktikum	4
BAB II.....	5
PEMBAHASAN	5
2.1 Pengertian Queue.....	5
2.2 Alat dan Bahan.....	5
2.3 Langkah langkah	6
BAB III	24
KESIMPULAN.....	24
3.1 Ringkasan Hasil Praktikum	24
3.2 Saran Pengembangan.....	24
DAFTAR PUSTAKA	25

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemograman dan struktur data, pengelolaan data yang teratur sangat penting agar proses kerja sistem menjadi efisien. Salah satu struktur data yang sering digunakan Adalah queue (antian). Queue digunakan untuk menyimpan data dengan prinsip FIFO (First In First Out), yaitu data yang pertama masuk akan menjadi data pertama yang keluar.

Konsep Queue banyak diterapkan dalam kehidupan sehari hari, seperti antrian di kasir, antrian kendaraan di gerbang tol, antrian cetak printer, dan sistem pelayanan pelanggan. Oleh karena itu, praktikum mengenai queue diperlukan agar mahasiswa memahami cara kerja, implementasi, serta penerapan queue dalam pemograman.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut :

1. Memahami konsep dasar struktur data Queue.
2. Mengetahui prinsip kerja FIFO (First In First Out).
3. Mempelajari implementasi queue menggunakan array maupun linked list.
4. Melatih kemampuan membuat program queue dalam Bahasa pemograman.
5. Mengetahui operasi operasi dasar queue seperti enqueue, dequeue, peek/front, rear, isEmpty dan Infull.
6. Mengembangkan logika pemograman dalam menyelesaikan masalah menggunakan queue.

1.3 Manfaat Praktikum

Adapun manfaat dari praktikum Queue ini adalah sebagai berikut:

1. Mahasiswa dapat memahami pengelolaan data secara berurutan.
2. Menambah kemampuan dalam membuat program struktur data.

3. Membantu memahami dalam membuat program struktur data.
4. Menjadi dasar untuk mempelajari algoritma dan struktur data lanjutan.
5. Meningkatkan kemampuan analisis dan pemecahan masalah dalam pemrograman.

BAB II

PEMBAHASAN

2.1 Pengertian Queue

Queue Adalah struktur data linear yang bekerja dengan prinsip FIFO (First In First Out), artinya elemen yang pertama dimasukkan ke dalam queue akan menjadi elemen yang pertama yang dikeluarkan.

Operasi dasar pada queue:

- Enqueue: menambahkan data ke belakang antrian
- Dequeue: menghapus data dari depan antrian.
- Front/Ppeek: melihat data terdepan tanpa menghapusnya
- Rear: melihat data terakhir.
- isEmpty: mengecek apakah queue kosong.
- isFull: mengecek apakah queue penuh (jika menggunakan array statis).

2.2 Alat dan Bahan

1. Laptop
2. Aplikasi aclipse
3. Paket data yang stabil

2.3 Langkah-langkah

```
pekan4_2511533015;  
  
public class QueueArray_2511533015{  
    int front_3015, rear_3015, size_3015;  
    int capacity_3015;  
    int array_3015[];  
  
    public QueueArray_2511533015 (int capacity) {  
        this.capacity_3015 = capacity;  
        front_3015 = this.size_3015 = 0;  
        rear_3015 = capacity - 1;  
        array_3015 = new int [this.capacity_3015];  
    }  
}
```

```

boolean inFull_3015 (QueueArray_2511533015 queue) {
    return (queue.size_3015 == queue.capacity_3015);
}
boolean isEmpty_3015 (QueueArray_2511533015 queue) {
    return (queue.size_3015 == 0);
}
void enqueue_3015 (int item) {
    if (inFull_3015 (this))
        return;
    this.rear_3015 = (this.rear_3015 + 1) % this.capacity_3015;
    this.array_3015 [this.rear_3015] = item;
    this.size_3015 = this.size_3015 +1;
    System.out.println (item +" enqueued to queue");
}
int dequeue_3015 () {
    if (isEmpty_3015 (this))
        return Integer.MIN_VALUE;
    int item = this.array_3015 [this.front_3015];
    this.front_3015 = (this.front_3015 + 1)% this.capacity_3015;
    this.size_3015 = this.size_3015 - 1;
    return item;
}
int front_3015 () {
    if (isEmpty_3015 (this))
        return Integer .MIN_VALUE;
    return this.array_3015 [this.front_3015];
}
int rear_3015 () {
    if (isEmpty_3015 (this))
        return Integer.MIN_VALUE;
    return this.array_3015 [this.rear_3015];
}
//mencetak elemen antrian
void display_3015 () {
    int i;
    if (front_3015 == rear_3015) {
        System.out.printf ("\nAntrian Kosong\n");
        return;
    }
    //kunjungi dari belakang dan cetak
    for (i = front_3015; i < rear_3015; i++) {
        System.out.printf (" %d <--", array_3015 [i]);
    }
    return;
}
}

```

a. Class QueueArray_2511533015

1. Deklarasi variable

```
int front_3015, rear_3015, size_3015;
int capacity_3015;
int array_3015[];
```

- Penjelasan:
Front_3015 = posisi depan antrian
Rear_3015 = posisi belakang antrian
Size_3015 = jumlah data saat ini
Capacity_3015 = kapasitas maksimum queue
Array_3015[] = tempat menyimpan data

2. Constructor

```
public QueueArray_2511533015(int capacity)
```

- Penjelasan:
Front = 0
Size = 0
Rear = capacity – 1
Membuat array sesuai kapasitas

3. Cek queue penuh

```
boolean inFull_3015(...)
```

- Penjelasan:
Mengembalikan nilai true jika queue sudah penuh

4. Cek queue kosong

```
boolean isEmpty_3015(...)
```

- Penjelasan:
mengembalikan nilai true jika queue kosong

5. Menambah data (enqueue)

```
void enqueue_3015(int item)
```

- Penjelasan:
Menambahkan data ke belakang queue
Langkah:
-cek penuh atau tidak
-rear maju satu Langkah
-simpan data
-size bertambah
Contoh output:
10 enqueued to queue

6. Menghapus data (dequeue)

```
int dequeue_3015()
```

- Penjelasan:
Menghapus data paling depan
Langkah:
-Cek kosong atau tidak
-Ambil data depan
-front maju satu Langkah

-size berkurang

7. Melihat data depan

int front_3015()

- Penjelasan:

Menampilkan elemen paling depan tanpa menghapus

8. Melihat data belakang

int rear_3015()

- Penjelasan:

Menampilkan elemen paling belakang

9. Menampilkan isi queue

void display_3015()

- Penjelasan:

Menampilkan semua isi antrian dari depan ke belakang

Contoh:

10 <-- 20 <-- 30 <--

```
package pekan4_2511533015;

public class QueueArrayDriver_2511533015 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        QueueArray_2511533015 queue_3015 = new QueueArray_2511533015
(1000);
        queue_3015.enqueue_3015 (10);
        queue_3015.enqueue_3015 (20);
        queue_3015.enqueue_3015 (30);
        queue_3015.enqueue_3015 (40);
        System.out.println ("Item di depan "+ queue_3015.front_3015());
        System.out.println ("Item paling belakang "+
queue_3015.rear_3015());
        System.out.println ("tampilan queue");
        queue_3015.display_3015();
        System.out.println();
        System.out.println (queue_3015.dequeue_3015 () + "dihapus dari
queue");
        System.out.println ("Item di depan:" + queue_3015.front_3015());
        System.out.println ("Item di belakang:" +
queue_3015.rear_3015());
        System.out.println ("Tampilan queue setelah satu fdata dihapus");
        queue_3015.display_3015();

    }
}
```

- b. Class QueueArrayDriver_2511533015
1. Membuat objek queue
`QueueArray_2511533015 queue_3015 = new QueueArray_2511533015(1000);`
 - Penjelasan:
Membuat queue dengan kapasitas 1000 data
 2. Menambahkan data ke queue
`queue_3015.enqueue_3015(10);`
`queue_3015.enqueue_3015(20);`
`queue_3015.enqueue_3015(30);`
`queue_3015.enqueue_3015(40);`
 - Penjelasan:
Data dimasukkan berurutan:
10, 20, 30, 40
Urutan queue:
10 ← 20 ← 30 ← 40
 3. Menampilkan data depan
`System.out.println("Item di depan "+ queue_3015.front_3015());`
 - Penjelasan:
Menampilkan data paling depan:
10
 4. Menampilkan data belakang
`System.out.println("Item paling belakang "+ queue_3015.rear_3015());`
 - Penjelasan:
Menampilkan data paling belakang:
40
 5. Menampilkan isi queue
`queue_3015.display_3015();`
-output:
10 ← 20 ← 30 ← 40 ←
 6. Menghapus data depan
`System.out.println(queue_3015.dequeue_3015() + " dihapus dari queue");`
 - Penjelasan:
Data pertama (10) dihapus karena queue memakai sistem FIFO
Queue menjadi:
20 ← 30 ← 40 ←
 7. Menampilkan queue setelah hapus
front = 20
rear = 40
-isi queue:
20 ← 30 ← 40 ←

```

package pekan4_2511533015;
import java.util.Queue;
public class QueueLinkedList_2511533015 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Queue<Integer> q_3015= new LinkedList<>();
        //tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
        for (int i = 0; i < 6; i++)
            q_3015.add(i);
        //menampilkan isi antrian.
        System.out.println ("Elemen Antrian " + q_3015);
        //untuk menghapus kepala antrian.
        int hapus = q_3015.remove();
        System.out.println ("Hapus elemen = " + hapus);
        System.out.println (q_3015);
        //untuk melihat antrian terdepan
        int depan = q_3015.peek();
        System.out.println ("Kepala Antrian = " + depan);
        int banyak = q_3015.size ();
        System.out.println ("Size Antrian = " + banyak);
    }
}

```

c. Class QueueLinkedList_2511533015

1. Import Library

```

import java.util.Queue;
import java.util.LinkedList;

```

- Penjelasan:

Queue = interface antrian

LinkedList: implementasi queue menggunakan list

2. Membuat queue

```

Queue<Integer> q_3015 = new LinkedList<>();

```

- Penjelasan:
- Membuat queue bertipe integer (bilangan bulat)

3. Menambahkan data ke queue

```

for (int i = 0; i < 6; i++)
    q_3015.add(i);

```

- Penjelasan:

Menambahkan angka 0 sampai 5 ke queue

-isi queue:

[0, 1, 2, 3, 4, 5]

4. Menampilkan isi queue

```

System.out.println("Elemen Antrian " + q_3015);

```

Output:

Elemen Antrian [0, 1, 2, 3, 4, 5]

5. Menghapus data depan

```

int hapus = q_3015.remove();

```

- Penjelasan:

Menghapus elemen paling depan yaitu = 0

-queue menjadi:
[1, 2, 3, 4, 5]

6. Melihat elemen depan
int depan = q_3015.peek();
 - Penjelasan:
Melihat data paling depan tanpa menghapus
 - hasil:
1
7. Menampilkan jumlah data
int banyak = q_3015.size();
 - Penjelasan:
Menghitung jumlah elemen queue
 - hasil:
5

```
reservepackage pekan4_2511533015;
import java.util.LinkedList;
public class ReverseData_2511533015 {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Queue<Integer> q_3015 = new LinkedList<Integer>();
        q_3015.add(1);
        q_3015.add(2);
        q_3015.add(3); // [1, 2, 3]
        System.out.println("sebelum reverse" + q_3015);
        Stack<Integer> s_3015 = new Stack<Integer>();
        while (!q_3015.isEmpty()) { // Q -> S
            s_3015.push(q_3015.remove());
        }
        while (!s_3015.isEmpty()) { // S -> Q //
            q_3015.add(s_3015.pop());
        }
        System.out.println("sesudah reverse= " + q_3015); // [3,2,1]
    }
}
```

- d. Class ReverseData_2511533015
 1. Import library
import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;
 - Penjelasan:
Digunakan untuk membuat Queue dan Stack
 2. Membuat queue
Queue<Integer> q_3015 = new LinkedList<Integer>();
 - Penjelasan:
Membuat queue bertipe angka
 3. Menambahkan data

- ```

q_3015.add(1);
q_3015.add(2);
q_3015.add(3);

```
- isi queue:  
[1, 2, 3]
4. Menampilkan sebelum reverse  

```
System.out.println("sebelum reverse" + q_3015);
```

 -output:  
 sebelum reverse [1, 2, 3]
  5. Pindah queue ke stack  

```
while (!q_3015.isEmpty()) {
 s_3015.push(q_3015.remove());
}
```

    - Penjelasan:  
 Data dipindah:  
 -remove 1 → push ke stack  
 -remove 2 → push ke stack  
 -remove 3 → push ke stack  
 Stack menjadi:  
 [1, 2, 3]
  6. Pindah stack ke queue  

```
while (!s_3015.isEmpty()) {
 q_3015.add(s_3015.pop());
}
```

    - Penjelasan:  
 Karena stack LIFO:  
 -pop 3 → masuk queue  
 -pop 2 → masuk queue  
 -pop 1 → masuk queue  
 Queue sekarang:  
 [3, 2, 1]
  7. Menampilkan hasil  

```
System.out.println("sesudah reverse= " + q_3015);
```

 -output:  
 sesudah reverse= [3, 2, 1]

```

1 package pekan4_2511533015;
2 import java.util.Iterator;
3
4 public class IterasiQueue_2511533015 {
5
6 public static void main(String[] args) {
7 // TODO Auto-generated method stub
8 Queue<String> q_3015= new LinkedList <>();
9
10 q_3015.add ("Praktikum");
11 q_3015.add ("Struktur");
12 q_3015.add ("Data");
13 q_3015.add ("Dan");
14 q_3015.add ("Algoritma");
15 Iterator<String> iterator = q_3015.iterator ();
16 while (iterator.hasNext ()) {
17 System.out.print (iterator.next () + " ");
18 }
19 }
20 }

```

e. Class IterasiQueue\_2511533015

1. Import library

```

import java.util.Iterator;
import java.util.Queue;
import java.util.LinkedList;

```

- Penjelasan:  
Queue: struktur data antrian  
LinkedList: implementasi queue  
Iterator: alat untuk membaca satu per Satu

2. Membuat queue

```
Queue<String> q_3015 = new LinkedList<>();
```

- Penjelasan:  
Membuat queue bertipe string

3. Menambahkan data ke queue

```

q_3015.add("Praktikum");
q_3015.add("Struktur");
q_3015.add("Data");
q_3015.add("Dan");
q_3015.add("Algoritma");

```

-isi queue:  
[Praktikum, Struktur, Data, Dan, Algoritma]

4. Membuat iterator

```
Iterator<String> iterator = q_3015.iterator();
```

- Penjelasan:  
Digunakan untuk membaca elemen queue dari depan ke belakang

5. Menampilkan isi queue

```
while (iterator.hasNext()) {
 System.out.print(iterator.next() + " ");
}
```

- Penjelasan:
  - hasNext() = mengecek apakah masih ada data berikutnya
  - next() = mengambil data selanjutnya

## BAB III

### KESIMPULAN

#### 3.1 Ringkasan Hasil Praktikum

Berdasarkan praktikum yang dilakukan, queue berhasil diimplementasikan menggunakan program sederhana. Program mampu menjalankan operasi enqueue untuk menambahkan data ke antrian dan dequeue untuk mengambil data dari antrian sesuai prinsip FIFO.

Mahasiswa juga memahami bagaimana posisi front dan rear bekerja dalam queue. Saat data ditambahkan, rear akan bekerja ke belakang sedangkan saat data dihapus, front akan maju ke depan. Dari hasil praktikum, queue terbukti efektif digunakan untuk pengelolaan data yang membutuhkan sistem antrian.

#### 3.2 Saran Pengembangan

1. Mengembangkan queue menggunakan circular queue agar memori lebih efisien.
2. Menggunakan linked list queue agar ukuran queue dinamis.
3. Membuat simulasi queue pada kasus nyata, seperti antiran rumah sakit atau bank.
4. Menambahkan tampilan GUI agar program lebih interaktif.
5. Menggabungkan queue dengan algoritma lain untuk sistem yang lebih kompleks.

#### 3.3 Kesimpulan

Praktikum queue memberikan pemahaman penting mengenai struktur data antrian dengan prinsip FIFO. Queue sangat berguna dalam berbagai aplikasi nyata dan menjadi dasar penting dalam pembelajaran struktur data serta pemrograman lanjutan.

# TUGAS

## 1. Kode

```
package tugaspekand_2511533015;
import java.util.LinkedList;

public class Queue_2511533015 {

 public static void main(String[] args) {

 Scanner input_3015 = new Scanner(System.in);
 Queue<String> antrian_3015 = new LinkedList<>();

 int pilih_3015;

 do {
 System.out.println("=== PROGRAM ANTRIAN LOKET ===");
 System.out.println("1. Tambah Antrian");
 System.out.println("2. Hapus Antrian");
 System.out.println("3. Tampilkan Antrian");
 System.out.println("4. Reverse");
 System.out.println("5. Keluar");
 System.out.print("Pilih menu: ");
 pilih_3015 = input_3015.nextInt();
 input_3015.nextLine();

 switch (pilih_3015) {

 case 1:
 System.out.print("\nMasukkan nama pelanggan: ");
 String nama_3015 = input_3015.nextLine();

 antrian_3015.add(nama_3015);
 System.out.println("Data berhasil ditambahkan ke antrian\n");
 break;

 case 2:
 if (antrian_3015.isEmpty()) {
 System.out.println("\nAntrian kosong\n");
 } else {
 String hapus_3015 = antrian_3015.remove();
 System.out.println("\n" + hapus_3015 + " telah dilayani\n");
 }
 break;

 case 3:
 if (antrian_3015.isEmpty()) {
 System.out.println("\nAntrian kosong\n");
 } else {
 System.out.println("\nIsi antrian:");
 int no_3015 = 1;
 for (String data_3015 : antrian_3015) {
 System.out.println(no_3015 + ". " + data_3015);
 no_3015++;
 }
 System.out.println();
 }
 break;

 case 4:
 if (antrian_3015.isEmpty()) {
 System.out.println("\nAntrian kosong\n");
 } else {
 Stack<String> stack_3015 = new Stack<>();

 while (!antrian_3015.isEmpty()) {
 stack_3015.push(antrian_3015.remove());
 }

 while (!stack_3015.isEmpty()) {
 antrian_3015.add(stack_3015.pop());
 }

 System.out.println("\nIsi antrian:");
 int no_3015 = 1;
 for (String data_3015 : antrian_3015) {
 System.out.println(no_3015 + ". " + data_3015);
 no_3015++;
 }
 System.out.println();
 }
 break;

 case 5:
 System.out.println("\nProgram selesai");
 break;
 }
 }
 }
}
```

```

 no_3015++;
 }
 System.out.println();
}
break;

case 4:
 if (antrian_3015.isEmpty()) {
 System.out.println("\nAntrian kosong\n");
 } else {
 Stack<String> stack_3015 = new Stack<>();

 while (!antrian_3015.isEmpty()) {
 stack_3015.push(antrian_3015.remove());
 }

 while (!stack_3015.isEmpty()) {
 antrian_3015.add(stack_3015.pop());
 }

 System.out.println("\nIsi antrian:");
 int no_3015 = 1;
 for (String data_3015 : antrian_3015) {
 System.out.println(no_3015 + ". " + data_3015);
 no_3015++;
 }
 System.out.println();
 }
 break;

case 5:
 System.out.println("\nProgram selesai");
 break;

default:
 System.out.println("\nPilihan tidak valid\n");
}

} while (pilih_3015 != 5);

input_3015.close();
}

```

## 2. Penjelasan kode

### a. Variable utama

front\_3015, rear\_3015, size\_3015

capacity\_3015

array\_3015[]

➔ Front\_3015 = posisi data paling depan

➔ Rear\_3015 = posisi data paling belakang

➔ Size\_3015 = jumlah data dalam queue

➔ Capacity\_3015 = kapasitas maksimal queue

➔ Array\_3015[] = tempat menyimpan data

### b. Constructor

public QueueArray\_2511533015(int capacity)

➔ Digunakan untuk membuat queue baru sesuai kapasitas yang diinput

-front = 0

-size = 0

-rear = capacity = 1

Membuat array sesuai kapasitas

### c. Cek queue penuh

boolean inFull\_3015(...)

➔ Mengembalikan nilai true jika queue sudah penuh

### d. Cek queue kosong

boolean isEmpty\_3015(...)

➔ Mengembalikan nilai true jika queue kosong

### e. Menambah data (enqueue)

void enqueue\_3015(int item)

➔ Menambahkan data ke belakang queue

Langkah:

- cek penuh atau tidak
- rear maju satu Langkah
- simpan data
- size bertambah

Contoh output =

10 enqueued to queue

f. Menghapus data (dequeue)

int dequeue\_3015()

➔ Menghapus data paling depan

Langkah:

- cek kosong atau tidak
- ambil data depan
- front maju satu langkah
- size berkurang

g. melihat data depan

int front\_3015()

➔ Menampilkan elemen paling depan tanpa menghapus

h. Melihat data belakang

int rear\_3015()

➔ Menampilkan elemen paling belakang

i. Menampilkan isi queue

void display\_3015()

➔ Menampilkan semua isi antrian dari depan ke belakang

Contoh:

10 <-- 20 <-- 30 <--

3. Output

```

Queue_2511533015 [Java Application] C:\Users\user\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1

Masukkan nama pelanggan: aras
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3

Isi antrian:
1. aras

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1

Masukkan nama pelanggan: shofwa
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3

Isi antrian:
1. aras
2. shofwa

=== PROGRAM ANTRIAN LOKET ===

```